

TP2

Introduction au C++

Programmation Orientée Objet

Présentation

Le but de ce TP est de réaliser une classe structure de pile traitant des données de type `char`. La structure de pile sera implantée sous forme d'une classe, appelée `PiledChar`.

Contraintes et Remise du TP

- La correction tiendra compte de la brièveté des fonctions que vous écrivez (évitez les fonctions de plus de 25 lignes) - n'hésitez pas à découper une fonction en plusieurs sous-fonctions plus courtes.
- Vos classes et méthodes doivent être documentées.
- Pensez à compiler avec les flags `-Wall -Wextra -std=c++17` pour avoir un code qui soit le plus qualitatif possible.
- Les noms de classe commencent par une majuscule.
- Les noms de méthodes et d'attributs commencent par une minuscule.
- Organisez votre code avec des fichiers d'en-tête (.h) et des fichiers d'implémentation (.cpp) pour chaque classe.
- Utilisez des pointeurs intelligents (smart pointers) lorsque vous gérez de la mémoire dynamique.
- Les TP sont à réaliser en binômes, et à rendre dans le dossier dédié sur le Moodle InfoRob. Vous avez 4h pour faire les TP, et le rendu pris en compte sera le dernier déposé avant minuit.

Exercice 1: Partie 1 – Déclaration de la classe et constructeurs (6 points)

Créer trois fichiers : `tp2ex1.cpp`, `piledchar.h` et `piledchar.cpp`. Les fichiers `piledchar.h` et `piledchar.cpp` contiennent respectivement la déclaration de la classe `PiledChar` et les définitions des méthodes de la classe. Le fichier `tp2ex1.cpp` contient la fonction `main()` (chaque fichier .cpp effectue l'inclusion du fichier .h afin d'apporter au compilateur les informations relatives au contenu de la classe).

1. 3 points Écrire la déclaration de classe de `PiledChar` dans le fichier `piledchar.h`, en tenant compte des informations suivantes. La classe contient trois données membres privées : deux entiers strictement positifs, nommés `mMax` et `mSommet`, et un pointeur sur `char` nommé `mPile`. La donnée membre `mMax` contient la taille de la pile créée. La donnée membre `mSommet` indique le numéro de la case vide dans laquelle on pourra empiler le prochain caractère. Le pointeur `mPile` désigne le tableau de caractères, alloué dynamiquement.
2. 2 points Déclarer et écrire les différents constructeurs de la classe. La taille maximale de la pile sera par défaut **100**.
3. 1 point Définir aussi le constructeur de copie.

Exercice 2: Partie 2 – Méthodes et utilisation (7 points)

1. 1 point Écrire le destructeur, chargé de libérer les ressources allouées par chaque instance de classe.
2. 1 point Écrire une méthode `CompterElements()` qui retourne un entier positif correspondant au nombre d'éléments actuellement présents dans la pile.
3. 1 point Écrire la méthode `AfficherPile()` qui affiche entre des `[` et `]` les éléments actuellement présents dans la pile.
4. 1 point Écrire une méthode `EmpilerElem()` qui prend un caractère en paramètre et le place sur le dessus de la pile.
5. 1 point Écrire une méthode `DesempilerElem()` qui retourne le caractère du dessus de la pile.
6. 2 points Écrire dans le fichier `tp2ex1.cpp` la fonction `main()`. Déclarer un tableau de caractères et demandez à l'utilisateur d'entrer un mot au clavier. On utilisera la méthode `get()` associée à l'instruction `cin` sans oublier les trois paramètres

suivants : le tampon à remplir, le nombre maximal de caractères à lire et le délimiteur désignant la fin de la saisie. Par défaut, le délimiteur de saisie est le caractère newline ' \n '. Le `cin` rajoute automatiquement le caractère ' \0 ' à la fin de la chaîne de caractères. Les lignes suivantes illustrent le processus :

```
char buffer[80];
cin.get( buffer, 79 );
cout << "chaîne : " << buffer << endl;
```

Remarque : le caractère délimiteur n'est pas récupéré. Il reste donc dans le flux et sera retiré lors d'un autre appel ultérieur à `cin`, provoquant des effets indésirables. On peut éviter cela en utilisant la méthode `cin.getline(buffer, 79)` à la place de `cin.get(buffer, 79)`.

Créer une instance de la classe `PiledChar`. Le programme lit l'entrée clavier, découpe ensuite la chaîne de caractères lettre par lettre et place chaque lettre dans la pile. Si un caractère de fin est rencontré avant le nombre maximal de caractères autorisés, un caractère fin de chaîne est rajouté.

Exercice 3: Partie 3 – Fonctions auxiliaires (5 points)

Pour surveiller l'évolution de la pile à chaque empilage, afficher le contenu de la pile après avoir empilé chaque lettre.

1. 2 points Écrire une fonction `afficheinverse()` recevant en paramètre une pile et qui dépile les lettres qui y sont contenues en les écrivant à l'écran au fur et à mesure. Cette fonction ne doit **pas** modifier le contenu de la pile déclarée dans le `main()`. Utiliser cette fonction pour écrire à l'envers le message tapé par l'utilisateur.
2. 2 points Écrire une fonction `inversemajuscule()` qui reçoit une instance de la classe `PiledChar` en paramètre, qui crée une nouvelle pile `nPile`, dépile les lettres une par une pour ensuite les rempiler en majuscule dans `nPile`.
3. 1 point Afin de vérifier le programme, ajouter dans chaque constructeur et destructeur l'affichage d'un message indiquant que la construction/destruction a lieu.

Exercice 4: Partie 4 – Généralisation par template (2 points)

La classe `PiledChar` que vous avez réalisée ne traite que des caractères. On souhaite la généraliser en une classe **template** `Pile<T>`, capable de stocker des éléments de n'importe quel type. On s'appuiera sur les *templates de classe* vus en cours (section 1.3).

1. 1 point Réécrire la classe `PiledChar` sous la forme d'un template de classe `Pile<T>`, en remplaçant le type `char` par le paramètre de type `T`. Adaptez en conséquence le fichier d'en-tête `pile.h` (rappel : l'implémentation des méthodes d'un template de classe doit rester dans le fichier `.h`). Les signatures des méthodes `EmpilerElem()`, `DesempilerElem()`, `CompterElements()` et `AfficherPile()` doivent être conservées, avec `T` à la place de `char`.
2. 1 point Dans la fonction `main()`, vérifier que votre template fonctionne correctement avec au moins deux types différents : une `Pile<char>` (comportement identique aux parties précédentes) et une `Pile<int>`. Pour la pile d'entiers, empiler les valeurs 1, 2, 3, afficher la pile, puis dépiler et afficher le sommet obtenu.

Remarque : on rappelle que lors de l'instanciation d'un template de classe, le compilateur génère automatiquement une version spécialisée du code pour chaque type utilisé. Ainsi, `Pile<char>` et `Pile<int>` sont deux classes distinctes générées à la compilation.